Inventory Sync

Shopify Inventory Sync - Process Manual
- Overview
- Configuratio
- Shared Pipeline
- Stage 1 - Initialize
- Stage 2 - Start Session
- Stage 3 - Fetch Inventory
  - Full Sync - Fetch
  - Incremer Sync - Fetch
  - Slab Calculati (both processe
- Stage 4 - Update Inventory
  - Step 4a - Persist Batch Requests
  - Step 4b - Process Pending Requests
- Stage 5 - Complete

Shopify Inventory Sync - Process Manual

Overview

Two inventory sync processes push stock quantities from the ERP system to Shopify. Both share the same pipeline structure and infrastructure but differ in which items they select on each run and how they determine the item identifier used to locate the corresponding Shopify product variant.

Process	Purpose	Typical Schedule
Full Sync	Pushes stock for every item in scope on every run	Initial onboarding, periodic reconciliation
Incremental Sync	Pushes stock only for items that changed since the last completed session	Frequent runs (every 15–30 minutes)

Configuration

Three configuration settings control how both processes identify and locate product variants in Shopify. They are independent of each other and can be changed without invalidating any previously cached data.

Setting	Values	Effect
`ItemsScope`	`AllItems`, `ChannelItems`	Which items are candidates for the run

Session
Shared Model - `ResolvedInv`
Database Tables
`channels`
`channel`
`channela`
`channel`
`channel`
`channela`

Setting	Values	Effect
`ItemIdentifierInWINBDS`	`SKU`, `ISBN`	Which local field is sent to Shopify as the search value
`ShopifyFieldForItemIdentifier`	`SKU`, `Barcode`	Which Shopify product variant field is searched against

The four identifier/field combinations are all fully supported:

`ItemIdentifierInWINBDS`	`ShopifyFieldForItemIdentifier`	Search Shopify
SKU	SKU	SKU co against SKU fie
SKU	Barcode	SKU co against Barcod
ISBN	SKU	ISBN va against SKU fie mercha stores I SKU)
ISBN	Barcode	ISBN va against Barcod

The local reference cache (`channelinventoryitemreferences`) is always keyed by the item's **SKU code**, regardless of which combination is configured. This means switching configuration never invalidates cached mappings and never forces redundant Shopify API calls.

Shared Pipeline

Every run - full or incremental - goes through the same five stages in order.

```
Initialize → Start Session → Fetch Inventory → Update Inventory →
Complete Session
```

A failure in any stage is logged against the session and the run stops. The next scheduled run starts fresh.

Stage 1 - Initialize

Before anything else, the system loads the runtime environment and prepares the channel for processing.

1. The database connection (`DataDb`), installation id, and stock location id are loaded from the application context.
2. The Shopify GraphQL API client is constructed from configuration (GraphQL URL, access token, location id). No network call is made yet.
3. The inventory slab configuration is loaded and converted into a typed list for use during stock calculation.
4. The three channel configuration values are read: `ItemsScope`, `ItemIdentifierInWINBDS`, and `ShopifyFieldForItemIdentifier`.
5. **Stuck session recovery** runs. The system checks whether a previous session for this channel is still marked as *InProgress* and has exceeded the maximum session duration (30 minutes). If so, that session is marked **Suspended** with a `suspendedat` timestamp. This clears the way for the new session and ensures any pending requests from the stuck session are picked up in the processing stage.

If initialization fails for any reason, the run stops silently with no session record created.

Stage 2 - Start Session

A new session row is inserted into `channelsessions` with status **Initiated** and the current timestamp. The generated session id (`CurrentSessionId`) is carried through all subsequent writes in this run.

If the insert fails, the error is logged and the run stops.

Stage 3 - Fetch Inventory

This stage selects which items to push and calculates the export quantity for each one. This is where Full and Incremental differ.

Full Sync - Fetch

Behaviour depends on `ItemsScope`:

- **`ChannelItems`** - all item ids explicitly mapped to the channel in `channelitem` are collected with no date filter. Every mapped item is a candidate on every full run.
- **`AllItems`** - all items in `itemhelpinfo` are candidates, with no channel mapping join.

Once the item ids are collected, the system joins them against `itemhelpinfo` and the ERP `stock` table to get the current available quantity at the configured stock location. Items with no stock record default to zero. Both the SKU (`uacode`) and ISBN (`COALESCE(isbn13, isbn10, barcode)`) are selected for every item regardless of which identifier is configured, so the batch payload always carries both values.

Incremental Sync - Fetch

The watermark is read first - the `MAX(startedat)` from `channelsessions` for this channel where status is **Completed**. If no completed session exists (first run), the watermark defaults to 7 days ago.

Two change sources are queried for the window from the watermark to now:

- **Stock changes** - the `timestamp` table is scanned for `stock` records whose `createdat` or `updatedat` falls in the window. The item id is extracted from the pipe-delimited `keyvalue` column (`installationid|stocklocationid|itemid`).
- **Mapping changes** - only when `ItemsScope = ChannelItems`. The `channelitemlog` table is scanned for Insert (`I`) or Delete (`D`) operations in the window. A newly mapped item needs its opening stock pushed; a deleted mapping resolves to zero through the slab rules. This source is skipped entirely when `ItemsScope = AllItems` because channel mapping events are irrelevant in that mode.

The combined set of changed item ids is then hydrated from `itemhelpinfo` and `stock`. The query path depends on `ItemsScope`:

- **`ChannelItems`** - joins through `channelitem` to restrict to mapped items. Item ids in the changed set that are no longer present in `channelitem` are treated as deleted and pushed with zero quantity.
- **`AllItems`** - queries `itemhelpinfo` directly; no `channelitem` join and no deleted-item zero-quantity pass.

Both `Sku` and `ISBN` are selected for every item in both paths.

Slab Calculation (both processes)

After the stock quantity is retrieved, the export quantity is calculated using the slab configuration.

- The slab list is scanned for a band that contains the available quantity.
- If a matching band is found, the export quantity is `floor((allowedPercentage / 100) × availableQty)`.
- If no slab is configured at all, the full available quantity is exported.
- If a slab exists but the quantity falls outside every defined band, zero is exported.

The result is a list of `InventoryItem` objects - each carrying an item id, SKU, ISBN, and calculated export quantity - passed directly to Stage 4.

Stage 4 - Update Inventory

This stage is identical for both processes. It has two sub-steps: persist requests, then process them.

Step 4a - Persist Batch Requests

The item list is split into fixed-size batches (configurable via `MaxItemsPerRequest`). Before batching, SKUs are looked up in `channelinventoryitemreferences` in chunks of 2000 to avoid SQL Server expression limits. Any SKU that already has a known Shopify inventory item GID is stamped onto the batch item. SKUs with no prior mapping are left with a null `InventoryItemId` to be resolved at dispatch time.

Each batch is serialised as JSON and saved as a `channelrequests` row with status **Pending**. All batch rows are written before any API call begins. A crash after writing but before dispatching leaves recoverable pending rows that are picked up by the next run.

Step 4b - Process Pending Requests

All pending and retriable `InventoryAdjustBulk` requests across all sessions are fetched in ascending id order. There is no session filter - requests left pending from a previous suspended session are processed alongside the current session's new requests.

Retriable means: status **Failed**, `retrycount < 3`, and `responsehttpstatus = 500`.

Per-Request Processing

For each request in order:

1. The request is immediately marked **Processing** before any external call, so a concurrent run skips it.
2. The payload is deserialised back to a list of batch items. Each item carries `ItemId`, `Sku`, `ISBN`, `Quantity`, and `InventoryItemId`.
3. **Resolve unresolved ids.** Any item still carrying a null `InventoryItemId` has the active identifier value (SKU code or ISBN, depending on `ItemIdentifierInWINBDS`) sent to the Shopify API. The API method called depends on `ShopifyFieldForItemIdentifier`:

   - `SKU` → `ResolveInventoryItemIdsBySKUField` - queries the `inventoryItems` GraphQL endpoint; returns `Sku` and `InventoryItemId` per match.
   - `Barcode` → `ResolveInventoryItemIdsByBarcodeField` - queries the `productVariants` GraphQL endpoint; returns `Sku`, `Barcode`, and `InventoryItemId` per match.

   Both methods use cursor-based pagination to handle large result sets.
4. **Match-back.** The Shopify response is keyed by the "search key" - `resolved.Barcode` when the Barcode field was searched, `resolved.Sku` when the SKU field was searched. This always equals `GetActiveIdentifier(batchItem)`

for the correct pair, making the join reliable across all four configuration combinations.

5. **Persist new mappings.** Newly resolved items are upserted into `channelinventoryitemreferences` keyed by the item's local **SKU code** - always, regardless of which identifier was used to find them. This ensures the cache remains stable and fully reusable after any configuration change. The upsert is performed in chunks of 1000 rows using `UPDATE ... FROM (VALUES ...) ... INNER JOIN` followed by `INSERT ... SELECT ... WHERE NOT EXISTS`, costing 2 SQL statements per chunk regardless of row count.
6. **Stamp resolved ids** back onto the in-memory batch items so the dispatch list is built from memory without an extra query.
7. Items whose identifier could not be resolved in Shopify are **silently dropped** from the dispatch list. They are not written to the inventory ledger, so the incremental watermark will not advance for them and they will be included again on the next run.
8. If every item in the batch is unresolvable, the request is marked **Failed** (HTTP 404) and the run moves to the next request.
9. The Shopify `inventorySetQuantities` mutation is called with the resolved items.
10. The API call - with accurate start and end timestamps and elapsed milliseconds - is written to `channelapilogs` for both success and failure outcomes. The log condition is `(!callSucceeded || _enableSuccessfulApiCallLogging)`, so successful calls are only logged when that flag is explicitly enabled.
11. The outcome is evaluated:

Outcome	Request Status	Ledger Written
Network or transport exception	Failed (HTTP 500)	No
GraphQL errors or null data	Failed (HTTP 500)	No
Shopify user errors in response	Failed (HTTP 400)	No
Full success	Completed (HTTP 200)	Yes - all dispatched items

1. For a successful dispatch, one ledger row per item is inserted into `channelinventoryupdatelogs` in a single transaction. All rows share the same `updatedat` timestamp set once before the loop, keeping the incremental watermark consistent across the batch.

Stage 5 - Complete Session

The session row is updated to status **Completed** with a `completedat` timestamp.

Shared Model - `ResolvedInventoryItem`

Defined in `Lspl.Winbds.Channels.Models`. Carries the raw Shopify field values returned by the resolution API alongside the resulting inventory item GID.

Property	Source	Notes
`Sku`	`node.sku`	Populated by both resol May contain an ISBN if stores it there.
`Barcode`	`node.barcode`	Populated only by `ResolveInventoryItemI` `null` when resolved vi endpoint. May contain a merchant stores it there

Property	Source	Notes
`InventoryItemId`	`node.id` / `node.inventoryItem.id`	Stable Shopify GID. Doe when the merchant edit fields.

Database Tables

`channelsessions`

Tracks each run of either sync process.

```
CREATE TABLE channelsessions (
    id            BIGINT         NOT NULL PRIM
ARY KEY,  -- NEXT ID
    channelid     BIGINT         NOT NULL,
    startedat     DATETIME       NOT NULL,
    status        BIGINT         NOT NULL,  --
0=Initiated, 1=InProgress, 2=Completed, 3=Sus
pended
    iserror       BIT            NOT NULL DEFA
ULT 0,
    errormessage NVARCHAR(MAX)  NULL,
    stacktrace   NVARCHAR(MAX)  NULL,
    completedat  DATETIME       NULL,
    suspendedat  DATETIME       NULL
);
```

`suspendedat` is set when a previous session is recovered by the stuck-session check. `completedat` is set when a session finishes normally. `MAX(startedat) WHERE status = Completed` is the watermark used by the incremental sync.

`channelrequests`

One row per batch of items. Carries the full item payload and tracks processing state.

```sql
CREATE TABLE channelrequests (
    id                  BIGINT          NOT NU
LL PRIMARY KEY,  -- NEXT ID
    sessionid           BIGINT          NOT NU
LL,
    channelid           BIGINT          NOT NU
LL,
    requesttype         BIGINT          NOT NU
LL,  -- InventoryAdjustBulk = specific enum v
alue
    createdat           DATETIME        NOT NU
LL,
    status              BIGINT          NOT NU
LL,  -- 1=Pending, 2=Processing, 3=Completed,
4=Failed
    requestpayload      VARBINARY(MAX)  NULL,
-- JSON batch, stored as UTF-8 bytes
    referenceid         VARCHAR(255)    NULL,
    retrycount          BIGINT          NOT NU
LL DEFAULT 0,
    processedat         DATETIME        NULL,
    responsehttpstatus BIGINT           NULL,
    responsecontent     VARBINARY(MAX)  NULL
);
```

`retrycount` is incremented on every failed attempt. Requests with status **Failed**, `retrycount < 3`, and `responsehttpstatus = 500` are eligible for retry on the next run.

`channelapilogs`

One row per Shopify API call. Written for both success and failure outcomes.

```sql
CREATE TABLE channelapilogs (
    id                  BIGINT          NOT NU
LL PRIMARY KEY IDENTITY(1,1),
    channelid           BIGINT          NOT NU
LL,
    sessionid           BIGINT          NOT NU
LL,
    referenceid         BIGINT          NOT NU
LL,
    operation           VARCHAR(MAX)    NOT NU
LL,  -- e.g. "SetInventory"
    requestcurl         NVARCHAR(MAX)   NULL,
    httpstatuscode      BIGINT          NULL,
-- 200 on success, 400/500 on failure
    issuccessresponse   BIT             NULL,
    responsecontenttype VARCHAR(MAX)    NULL,
    responsecontent     NVARCHAR(MAX)   NULL,
    startedat           DATETIME        NOT NU
LL,
    completedat         DATETIME        NOT NU
LL,
    elapsedtimeinms     BIGINT          NOT NU
LL
);
```

`startedat` is captured immediately before the HTTP call; `completedat` immediately after. Successful calls are only logged when `EnableSuccessfulAPICallLogging` is enabled in configuration.

`channelinventoryupdatelogs`

The inventory push ledger. One row per successfully dispatched item per request. `MAX(startedat)` from `channelsessions WHERE status = Completed` — not this table — is the incremental watermark.

```sql
CREATE TABLE channelinventoryupdatelogs (
    id        BIGINT        NOT NULL PRIMARY
KEY IDENTITY(1,1),
    channelid BIGINT        NOT NULL,
    sessionid BIGINT        NOT NULL,
    requestid BIGINT        NOT NULL,
    itemid    BIGINT        NOT NULL,
    skucode   VARCHAR(255)  NOT NULL,  -- alw
ays the raw SKU, regardless of identifier con
fig
    pushedqty BIGINT        NOT NULL,
    updatedat DATETIME      NOT NULL
);
```

All rows written from a single batch share the same `updatedat` value, set once before the insert loop. `skucode` always stores the raw SKU code regardless of which identifier (`SKU` or `ISBN`) is configured — it is an audit field, not a lookup key.

`channelinventoryitemreferences`

Persistent cross-reference between ERP items and Shopify inventory item GIDs. Eliminates repeated resolution API calls for known items.

```sql
CREATE TABLE channelinventoryitemreferences (
    id          BIGINT        NOT NULL PRIMAR
Y KEY IDENTITY(1,1),
    channelid   BIGINT        NOT NULL,
    itemid      BIGINT        NOT NULL,
    sku         VARCHAR(255)  NOT NULL,
-- local SKU code; always the cache key
    referenceid VARCHAR(MAX)  NOT NULL,
-- Shopify GID, e.g. gid://shopify/InventoryI
tem/123
```

```
    CONSTRAINT uq_channelinventoryitemreferen
ces_channel_sku UNIQUE (channelid, sku)
);
```

The table is keyed by `(channelid, sku)` regardless of the `ItemIdentifierInWINBDS` or `ShopifyFieldForItemIdentifier` configuration. Consequences:

- Switching from SKU to ISBN mode (or vice versa) does not invalidate existing rows. The next run finds all previously cached GIDs immediately with no API calls.
- The Shopify inventory item GID is a stable internal identifier that does not change when a merchant edits the SKU or Barcode fields on a product variant.

Upsert strategy: `UPDATE ... FROM (VALUES ...) INNER JOIN` first; `INSERT ... SELECT ... WHERE NOT EXISTS` for rows with no match. Performed in chunks of 1000 rows - 2 SQL statements per chunk regardless of how many rows the chunk contains. Compatible with SQL Server 2008 and later.

Read strategy: `SELECT ... WHERE sku IN (...)` in chunks of 2000 to stay within SQL Server's expression services limit.

`channelauthinfo`

Stores the current Shopify OAuth access token and its validity window for each channel. One row per channel. Written by `ShopifyTokenManager` on every successful token refresh; read before every Shopify API call.

```
CREATE TABLE channelauthinfo (
    id          BIGINT         NOT NULL PRIMA
RY KEY IDENTITY(1,1),
  channelid   BIGINT         NOT NULL,
  accesstoken NVARCHAR(MAX)  NOT NULL,
  generatedon DATETIME       NOT NULL,
```

```
  expiredon   DATETIME       NULL
);
```

`expiredon` is nullable to handle legacy rows written before expiry tracking was introduced. When `expiredon` is `NULL`, `IsExpiredOrExpiringSoon` returns `false` and the stored token is treated as still valid.